Laboratorio 03

Sistema académico y refactor por módulos con Ports

Ingeniería de Software III

Proyecto demo-academico

Rama Git: Laboratorio_03

Repositorio: Jasu17/Lab01_ing_software_3

Este documento reúne respuestas HTTP capturadas y capturas de pantalla tomadas sobre la aplicación en ejecución. No incluye resultados simulados.

2. Entorno y forma de ejecución

El proyecto Maven está en DemoAcademicoApplication. Compila con Java 21 (release 21) y Spring Boot 3.2.5. En esta máquina JAVA_HOME apunta a JDK 11, así que Maven se ejecutó con JDK 25.0.2, que puede emitir bytecode 21.

Comandos, desde DemoAcademicoApplication:

```
$env:JAVA_HOME = "C:\Program Files\Java\jdk-25.0.2"
.\mvnw.cmd test
.\mvnw.cmd spring-boot:run
```

Swagger: http://localhost:8080/swagger-ui.html

Consola H2: http://localhost:8080/h2-console

JDBC: jdbc:h2:mem:demoacademico. Usuario sa. Contraseña vacía.

Las pruebas automáticas de la implementación terminaron con Tests run: 4, Failures: 0, Errors: 0. Incluyen MatriculaRulesTest y MatriculaArchitectureTest.

La Parte 1 se ejerció el 14 de octubre de 2026 contra el commit e2023f1, en el puerto 8081, antes del refactor. La Parte 2 se ejerció de nuevo el 18 de octubre de 2026 contra la rama Laboratorio_03, en el puerto 8080.

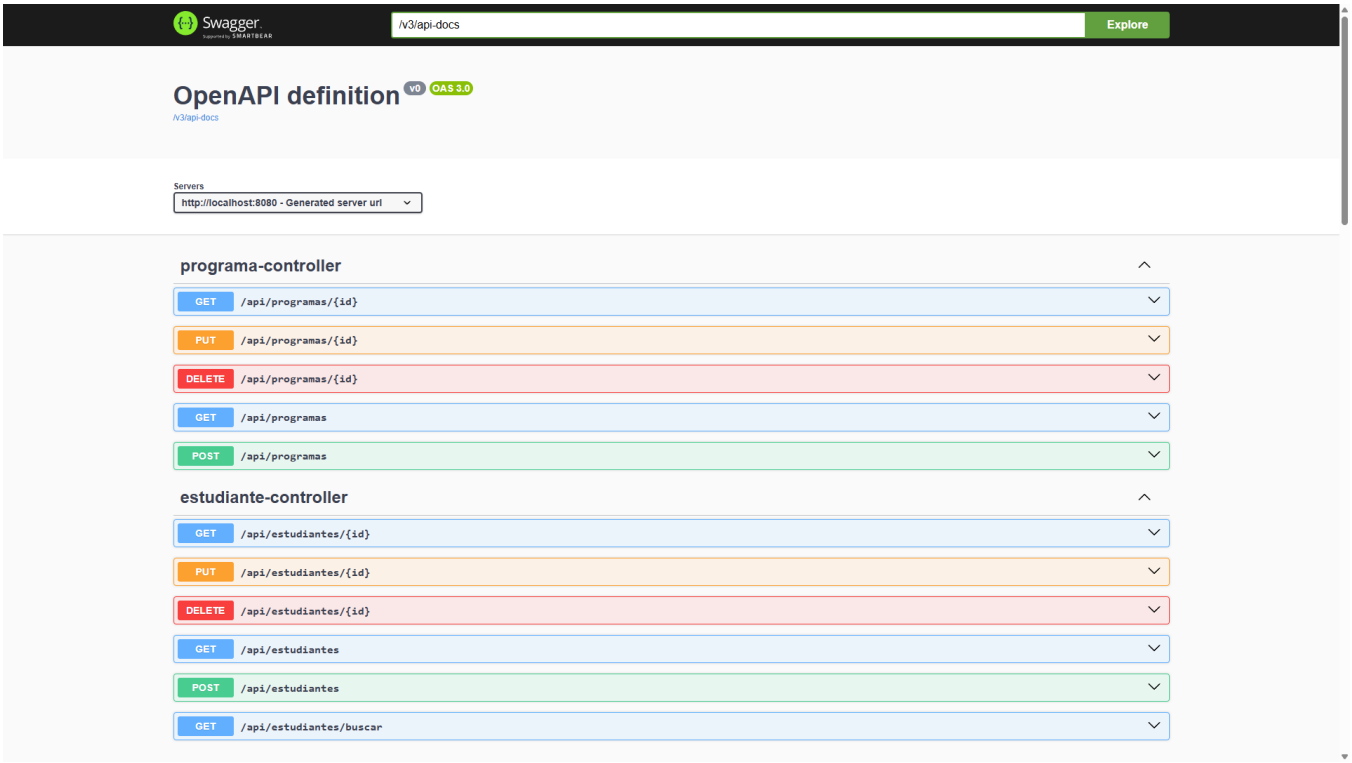


Figura 1. Swagger de la aplicación en ejecución, con los endpoints de programas y estudiantes.



Figura 2. Swagger muestra POST /api/matriculas, /api/grupos y /api/asignaturas.

Parte 1

3. Creación de un programa

POST /api/programas contra la Parte 1, commit e2023f1, puerto 8081. Resultado HTTP 201.

```
HTTP 201
{"success":true,"message":"Programa creado","data":{"id":1,"codigo":"PROG-ING","nombre":"Ingenieria de Software"},"timestamp":"2026-10-14T03:05:44.190409800Z"}
```

4. Creación de una asignatura

POST /api/asignaturas asociada al programa 1. Resultado HTTP 201.

```
HTTP 201
{"success":true,"message":"Asignatura creada","data":{"id":1,"codigo":"ASIG-IS3","nombre":"Ingenieria de Software III","creditos":3,"programaId":1,"programaNombre":"Ingenieria de Software"},"timestamp":"2026-10-14T03:05:44.246712300Z"}
```

5. Creación de un grupo con cupo 2

POST /api/grupos con cupoMax 2. Resultado HTTP 201.

```
HTTP 201
{"success":true,"message":"Grupo creado","data":{"id":1,"codigoGrupo":"G-IS3-01","cupoMax":2,"asignaturaId":1,"asignaturaNombre":"Ingenieria de Software III"},"timestamp":"2026-10-14T03:05:44.260832800Z"}
```

6. Primera matrícula exitosa

Primera matrícula del estudiante 1 en el grupo 1, con cupo todavía disponible. Resultado HTTP 201.

```
HTTP 201
{"success":true,"message":"Matrícula registrada","data":{"id":1,"estudianteId":1,"estudianteNombre":"Ana Uno","grupoId":1,"codigoGrupo":"G-IS3-01","fechaRegistro":"2026-10-14T03:05:44.315928500Z"},"timestamp":"2026-10-14T03:05:44.320932800Z"}
```

7. Segunda matrícula exitosa

Segunda matrícula, de otro estudiante, en el mismo grupo. Resultado HTTP 201.

```
HTTP 201
{"success":true,"message":"Matrícula registrada","data":{"id":2,"estudianteId":2,"estudianteNombre":"Luis Dos","grupoId":1,"codigoGrupo":"G-IS3-01","fechaRegistro":"2026-10-14T03:05:44.344930500Z"},"timestamp":"2026-10-14T03:05:44.345929200Z"}
```

8. Rechazo por duplicado

Se repitió la matrícula del estudiante 1 antes de llenar el cupo. El mensaje es de duplicado, no de cupo. Resultado HTTP 400.

```
HTTP 400
{"success":false,"message":"El estudiante ya está matriculado en ese grupo","data":null,"timestamp":"2026-10-14T03:05:44.335930Z"}
```

9. Rechazo por cupo

Un tercer estudiante distinto intentó entrar al grupo ya lleno. Resultado HTTP 400 por cupo.

```
HTTP 400
{"success":false,"message":"El grupo no tiene cupo disponible","data":null,"timestamp":"2026-10-14T03:05:44.354930100Z"}
```

Estudiante y grupo inexistentes

Estudiante 999999. Resultado HTTP 404.

```
HTTP 404
{"success":false,"message":"Estudiante no encontrado: 999999","data":null,"timestamp":"2026-10-14T03:05:44.359928700Z"}
```

Grupo inexistente en la Parte 1

POST con grupoId 999999 y un estudiante existente.

```
HTTP 404
{"success":false,"message":"Grupo no encontrado: 999999","data":null,"timestamp":"2026-10-14T03:05:44.364928700Z"}
```

9. Estructura modular final

Listado real de los fuentes bajo co.edu.demoacademico, después del refactor:

```
asignaturas\controller\AsignaturaController.java
asignaturas\dto\AsignaturaCreateDTO.java
asignaturas\dto\AsignaturaDTO.java
asignaturas\handler\AsignaturaHandler.java
asignaturas\model\Asignatura.java
asignaturas\repository\AsignaturaRepository.java
asignaturas\service\AsignaturaService.java
asignaturas\service\AsignaturaServiceImpl.java
common\api\ApiResponse.java
common\api\ErrorResponse.java
common\api\ResponseBuilder.java
common\controller\GlobalExceptionHandler.java
common\exception\BusinessException.java
common\exception\EmailDuplicadoException.java
common\exception\NotFoundException.java
config\DataSeeder.java
config\DataSeederAcademico.java
DemoAcademicoApplication.java
estudiantes\controller\EstudianteController.java
estudiantes\dto\EstudianteCreateDTO.java
estudiantes\dto\EstudianteDTO.java
estudiantes\dto\EstudianteUpdateDTO.java
estudiantes\handler\EstudianteHandler.java
estudiantes\model\Estudiante.java
estudiantes\port\EstudianteQueryPort.java
estudiantes\repository\EstudianteRepository.java
estudiantes\service\EstudianteService.java
estudiantes\service\EstudianteServiceImpl.java
grupos\controller\GrupoController.java
grupos\dto\GrupoCreateDTO.java
grupos\dto\GrupoDTO.java
grupos\handler\GrupoHandler.java
grupos\model\Grupo.java
grupos\port\GrupoQueryPort.java
grupos\repository\GrupoRepository.java
grupos\service\GrupoService.java
grupos\service\GrupoServiceImpl.java
matriculas\controller\MatriculaController.java
matriculas\dto\MatriculaCreateDTO.java
matriculas\dto\MatriculaDTO.java
matriculas\handler\MatriculaHandler.java
matriculas\model\Matricula.java
matriculas\repository\MatriculaRepository.java
matriculas\service\MatriculaService.java
matriculas\service\MatriculaServiceImpl.java
programas\controller\ProgramaController.java
programas\dto\ProgramaCreateDTO.java
programas\dto\ProgramaDTO.java
programas\dto\ProgramaUpdateDTO.java
programas\handler\ProgramaHandler.java
programas\model\Programa.java
programas\repository\ProgramaRepository.java
programas\service\ProgramaService.java
programas\service\ProgramaServiceImpl.java
```

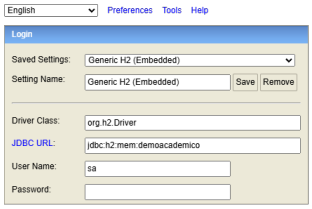


Figura 3. Consola H2 disponible en /h2-console, con la URL jdbc:h2:mem:demoacademico.

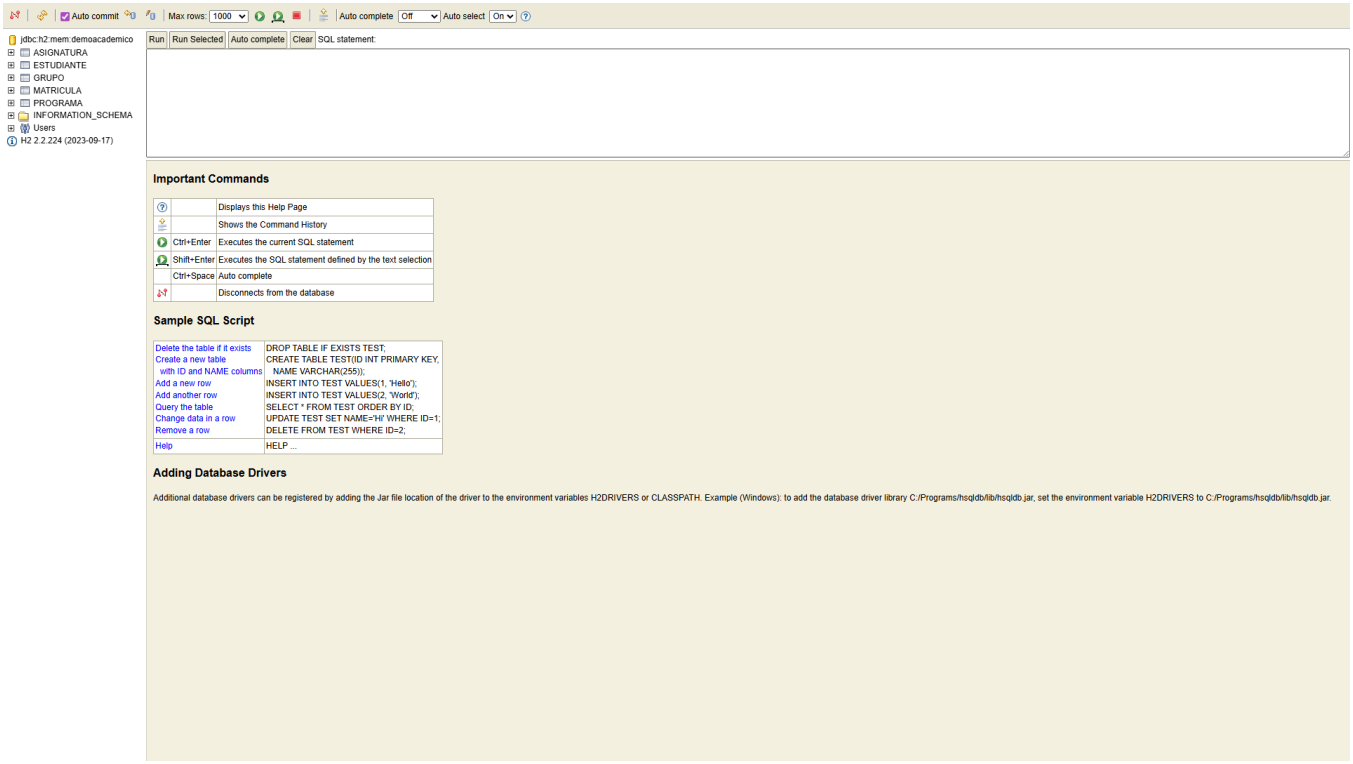


Figura 4. Sesión conectada a H2 después del guion de la Parte 2.

Conteos ejecutados en esa sesión H2

La consulta se envió a la consola H2 ya autenticada. La respuesta HTML real quedó en docs/evidencias/h2-conteos.html. Los conteos coinciden con el guion: un programa, una asignatura, un grupo, dos matrículas y tres estudiantes.

```
SELECT &#39;programa&#39; AS tabla, COUNT(*) AS total FROM programa UNION ALL SELECT &#39;asignatura&#39;, COUNT(*) FROM asignatura UNION ALL SELECT &#39;grupo&#39;, COUNT(*) FROM grupo UNION ALL SELECT &#39;matricula&#39;, COUNT(*) FROM matricula UNION ALL SELECT &#39;estudiante&#39;, COUNT(*) FROM estudiante;
| TABLA | TOTAL
```

```
| programa | 1
| asignatura | 1
| grupo | 1
| matricula | 2
| estudiante | 3
(5 rows, 1 ms)
```

10. Ports

EstudianteQueryPort, archivo real del repositorio:

```
package co.edu.demoacademico.estudiantes.port;

import co.edu.demoacademico.estudiantes.model.Estudiante;

public interface EstudianteQueryPort {
    Estudiante obtenerPorId(Long id);
}
```

GrupoQueryPort:

```
package co.edu.demoacademico.grupos.port;

import co.edu.demoacademico.grupos.model.Grupo;

public interface GrupoQueryPort {
    Grupo obtenerPorId(Long id);
}
```

11. MatriculaServiceImpl usa ports

El servicio de matrículas inyecta EstudianteQueryPort y GrupoQueryPort. Su único repositorio es MatriculaRepository. No importa EstudianteRepository ni GrupoRepository.

```
package co.edu.demoacademico.matriculas.service;

import co.edu.demoacademico.common.exception.BusinessException;
import co.edu.demoacademico.estudiantes.model.Estudiante;
import co.edu.demoacademico.estudiantes.port.EstudianteQueryPort;
import co.edu.demoacademico.grupos.model.Grupo;
import co.edu.demoacademico.grupos.port.GrupoQueryPort;
import co.edu.demoacademico.matriculas.model.Matricula;
import co.edu.demoacademico.matriculas.repository.MatriculaRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@Transactional
public class MatriculaServiceImpl implements MatriculaService {

    private final EstudianteQueryPort estudiantePort;
    private final GrupoQueryPort grupoPort;
    private final MatriculaRepository matriculaRepo;

    public MatriculaServiceImpl(EstudianteQueryPort estudiantePort,
                                GrupoQueryPort grupoPort,
                                MatriculaRepository matriculaRepo) {
        this.estudiantePort = estudiantePort;
        this.grupoPort = grupoPort;
        this.matriculaRepo = matriculaRepo;
    }

    @Override
    public Matricula matricular(Long estudianteId, Long grupoId) {
        Estudiante estudiante = estudiantePort.obtenerPorId(estudianteId);
        Grupo grupo = grupoPort.obtenerPorId(grupoId);

        if (matriculaRepo.existsByEstudianteIdAndGrupoId(estudianteId, grupoId)) {
            throw new BusinessException("El estudiante ya está matriculado en ese grupo");
        }
```

```
        long matriculados = matriculaRepo.countByGrupoId(grupoId);
        if (matriculados >= grupo.getCupoMax()) {
            throw new BusinessException("El grupo no tiene cupo disponible");
        }

        Matricula m = new Matricula();
        m.setEstudiante(estudiante);
        m.setGrupo(grupo);
        return matriculaRepo.save(m);
    }
}
```

Parte 2

12. Dos matrículas exitosas después del refactor

Misma secuencia sobre la aplicación ya modular, puerto 8080, el 18 de octubre de 2026. Primera matrícula, HTTP 201.

```
HTTP 201
{"success":true,"message":"Matrícula registrada","data":{"id":1,"estudianteId":1,"estudianteNombre":"Ana Uno","grupoId":1,"codigoGrupo":"G-IS3-01","fechaRegistro":"2026-10-18T00:02:07.068898800Z"},"timestamp":"2026-10-18T00:02:07.087168100Z"}
```

Segunda matrícula exitosa después del refactor

Segunda matrícula de otro estudiante. HTTP 201.

```
HTTP 201
{"success":true,"message":"Matrícula registrada","data":{"id":2,"estudianteId":2,"estudianteNombre":"Luis Dos","grupoId":1,"codigoGrupo":"G-IS3-01","fechaRegistro":"2026-10-18T00:02:07.100084600Z"},"timestamp":"2026-10-18T00:02:07.100084600Z"}
```

13. Error por cupo después del refactor

Tercer estudiante en el grupo con cupo 2. HTTP 400.

```
HTTP 400
{"success":false,"message":"El grupo no tiene cupo disponible","data":null,"timestamp":"2026-10-18T00:02:07.100084600Z"}
```

14. Error por duplicado después del refactor

Repetición del estudiante 1 mientras aún había cupo. HTTP 400 por duplicado.

```
HTTP 400
{"success":false,"message":"El estudiante ya está matriculado en ese grupo","data":null,"timestamp":"2026-10-18T00:02:07.089978400Z"}
```

Recursos inexistentes después del refactor

Los dos errores controlados se conservan.

```
HTTP 404
{"success":false,"message":"Estudiante no encontrado: 999999","data":null,"timestamp":"2026-10-18T00:02:07.100084600Z"}

HTTP 404
{"success":false,"message":"Grupo no encontrado: 999999","data":null,"timestamp":"2026-10-18T00:02:07.116008900Z"}
```

15. Resumen de verificaciones

- Compilación con release 21: correcta, mediante mvnw test.
- Pruebas automáticas: 4 ejecutadas, 0 fallos, 0 errores.
- Parte 1, HTTP: programa 201, asignatura 201, grupo con cupo 2 en 201.
- Parte 1, HTTP: dos matrículas 201, duplicado 400, cupo 400, inexistentes 404.
- Parte 2, HTTP: el mismo guion, con los mismos mensajes, después del refactor.
- Swagger: expone /api/programas, /api/asignaturas, /api/grupos, /api/matriculas y /api/estudiantes.
- H2: consola abierta y conteos programa=1, asignatura=1, grupo=1, matrícula=2, estudiante=3.
- Ports presentes e implementados por EstudianteServiceImpl y GrupoServiceImpl.
- MatriculaServiceImpl no depende de los repositorios ajenos. Lo comprueba también MatriculaArchitectureTest.
- No hay una captura de cada operación dentro del panel Try it out de Swagger. Los resultados de esas operaciones están en las respuestas HTTP guardadas, que son las que ejecutó el guion.